Projekt: Air Condition - System Monitorowania Jakości Powietrza

Opis Ogólny

System monitorowania jakości powietrza i warunków atmosferycznych z komunikacją radiową LoRa (SX1262). Urządzenie mierzy temperaturę, wilgotność, ciśnienie atmosferyczne oraz stężenie pyłów PM1.0/PM2.5/PM10. Dane wyświetlane są na ekranie OLED i mogą być przesyłane bezprzewodowo.

Lista Komponentów (BOM)

#	Element	Model	Interfejs	Napięcie	Uwagi
1	MCU	XIAO ESP32-S3 Sense	-	3.3V/5V USB	Główny kontroler
2	Moduł LoRa	Wio-SX1262	SPI (B2B)	3.3V	Komunikacja radiowa
3	Czujnik temp./wilg.	SHTC3 (DFRobot SEN0414)	I2C	3.3V	Adres: 0x70
4	Czujnik ciśnienia	BMP280 (DFRobot SEN0372)	I2C	3.3V	Adres: 0x77 lub 0x76
5	Wyświetlacz	OLED 0.96" 128x64 niebieski	I2C	3.3V-5V	Adres: 0x3C
6	Czujnik pyłu	PMS7003 + adapter IDC-microUSB (CH340G)	UART	5V	Logika 3.3V TTL, goldpiny: 5V/GND/RX/TX
7	Buzzer	Moduł z buzzerem aktywnym (zielony)	GPIO	3.3V	Sygnalizacja alarmów, bezpośrednio do GPIO
8	Zasilanie	Akumulator Li-Po 4000mAh	-	3.7V	Ładowanie przez USB
9	Przetwornica	Pololu S7V7F5 (step-up/step-down)	-	IN: 2.7-11.8V -> OUT: 5V 1A	Zasilanie 5V dla PMS7003
10	MOSFET N-channel	IRLZ44N (TO-220)	GPIO	V_DS: 55V, I_D: 47A, V_GS(th): 1-2V	Low-side switching zasilania peryferiów
11	Rezystory	10kΩ (THT)	-	-	Pull-down dla bramek MOSFET

Linki do Zakupu (Botland)

Element	Link
Pololu S7V7F5	https://botland.com.pl/przetwornice-step-updown-pololu/1696-pololu-s7v7f5-przetwornica-step-upstep-down-5v-1a-5904422359898.html

Element	Link
IRLZ44N (5szt)	https://botland.com.pl/n-mosfet/1950-n-mosfet-irlz44n-55v47a-tht-5szt-5904422373276.html
Adapter PMS7003 IDC-microUSB	https://botland.com.pl/czujniki-czystosci-powietrza/13445-adapter-idc-10pin-127mm-microusb-dla-czujnika-pms7003-5904422372293.html
Moduł z buzzerem aktywnym (zielony)	https://botland.com.pl/buzzery-generatory-dzwieku/4526-modul-z-buzzerem-aktywnym-z-generatorem-zielony-5904422300654.html

Ilości Elementów do Zakupu

Element	Ilość	Przeznaczenie	Status
IRLZ44N N-MOSFET	2 szt	Q1: low-side switch 5V (PMS7003+Pololu), Q2: low-side switch 3.3V (czujniki I2C+OLED)	**KUPIONY**
Rezystor 10kΩ	2 szt	R1, R2: pull-down bramek (domyślnie OFF)	DO ZAKUPU

Analiza Kompatybilności

Zgodność Napięć

Komponent	Zasilanie	Logika I/O	Kompatybilność z ESP32 (3.3V)
XIAO ESP32-S3	3.7V bat / 5V USB	3.3V	- (główny kontroler)
Wio-SX1262	3.3V	3.3V	**OK** - natywne 3.3V
SHTC3	1.62V-3.6V	3.3V	**OK** - natywne 3.3V
BMP280	1.71V-3.6V	3.3V	**OK** - natywne 3.3V
OLED 128x64	3.3V-5V	3.3V	**OK** - toleruje 3.3V
PMS7003	5V	3.3V TTL	**WYMAGA 5V** - przez przetwornicę
Buzzer aktywny	3.3V	-	**OK** - bezpośrednio do GPIO (<20mA)
Pololu S7V7F5	2.7-11.8V	-	**OK** - zasila PMS7003
IRLZ44N N-MOSFET	-	3.3V bramka	**OK** - logic-level, V_GS(th)=1-2V, pełne otwarcie przy 3.3V

Wynik Analizy

Wszystkie komponenty są kompatybilne z ESP32-S3 (3.3V logika).

WAŻNE: Problem Zasilania 5V przy Pracy Bateryjnej

> **Pin 5V na XIAO ESP32-S3 NIE DZIAŁA przy zasilaniu z baterii!**

> XIAO nie posiada wbudowanej przetwornicy boost (step-up). Pin 5V jest bezpośrednio połączony z USB VBUS - dostarcza 5V tylko gdy podłączone jest USB.
>
> | Źródło zasilania | Pin 5V | Pin 3V3 |
> |---|---|---|
> | USB-C | **5V** | 3.3V |
> | Bateria 3.7V | **0V (brak!)** | 3.3V |

Rozwiązanie: Przetwornica Pololu S7V7F5

Przetwornica step-up/step-down konwertuje napięcie z baterii (3.7V) na stabilne 5V dla PMS7003.

Specyfikacja Pololu S7V7F5:

- Napięcie wejściowe: 2.7V - 11.8V
- Napięcie wyjściowe: 5V (stałe)
- Prąd wyjściowy: do 1A (wystarczający dla PMS7003 ~100mA)
- Sprawność: ~90%
- Wymiary: 9.7mm x 10.7mm x 3mm

Uwagi:

- PMS7003 wymaga 5V zasilania - **MUSI być zasilany z przetwornicy Pololu**
- Moduł buzzera aktywnego (zielony) - **bezpośrednio do GPIO** (pobór ~10-15mA)
- OLED może pracować na 3.3V (z modułu DFRobot ma wbudowany regulator)

Schemat Połączeń

Pinout XIAO ESP32-S3 Sense



Tabela Połączeń

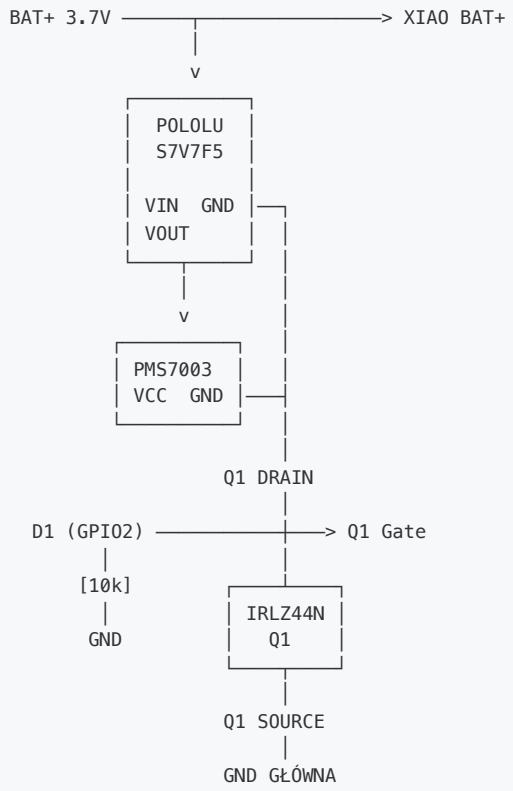
XIAO GPIO	Funkcja	Podłączony do	Kolor przewodu (sugestia)
I2C (wspólna magistrala)			
D4 (GPIO5)	SDA	SHTC3, BMP280, OLED	Niebieski
D5 (GPIO6)	SCL	SHTC3, BMP280, OLED	Żółty

XIAO GPIO	Funkcja	Podłączony do	Kolor przewodu (sugestia)
UART (PMS7003)			
D6 (GPIO43)	TX	PMS7003 RX	Zielony
D7 (GPIO44)	RX	PMS7003 TX	Biały
Sterowanie zasilaniem (MOSFETy)			
D1 (GPIO2)	PWR_5V_EN	Q1 bramka (przez R1) - zasilanie 5V	Fioletowy
D2 (GPIO3)	PWR_3V3_EN	Q2 bramka (przez R2) - zasilanie 3.3V peryferiów	Szary
Buzzer			
D0 (GPIO1)	BUZZER_OUT	Buzzer S (bezpośrednio)	Pomarańczowy
Przycisk wybudzenia			
D3 (GPIO4)	BUTTON_WAKE	Przycisk NO do GND (wbudowany pull-up)	Brązowy
Zasilanie stałe (zawsze włączone)			
3V3	3.3V	Rezystory pull-up MOSFETów	Czerwony
GND	GND	Wszystkie moduły	Czarny
SX1262 (B2B - wbudowane)			
GPIO41	NSS/CS	SX1262	(B2B)
GPIO7	SCK	SX1262	(B2B)
GPIO9	MOSI	SX1262	(B2B)
GPIO8	MISO	SX1262	(B2B)
GPIO42	RST	SX1262	(B2B)
GPIO40	BUSY	SX1262	(B2B)
GPIO39	DIO1/IRQ	SX1262	(B2B)
GPIO38	ANTSW	SX1262	(B2B)
Akumulator			
BAT+	+	Akumulator 3.7V (+)	Czerwony
BAT-	-	Akumulator 3.7V (-)	Czarny

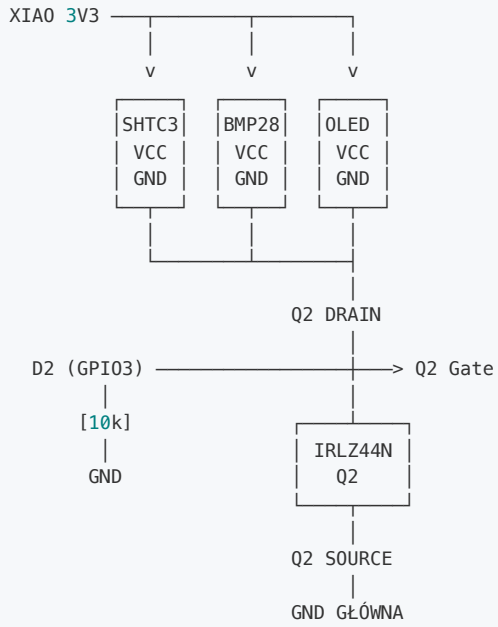
Schemat Połączeń na Płytce Uniwersalnej (Low-Side Switching)

UWAGA: GND Pololu i GND PMS7003 idą RAZEM do Q1 Drain!

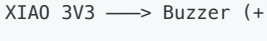
SEKCJA ZASILANIA 5V (Q1)
==========



SEKCJA I2C 3.3V (Q2)
==========

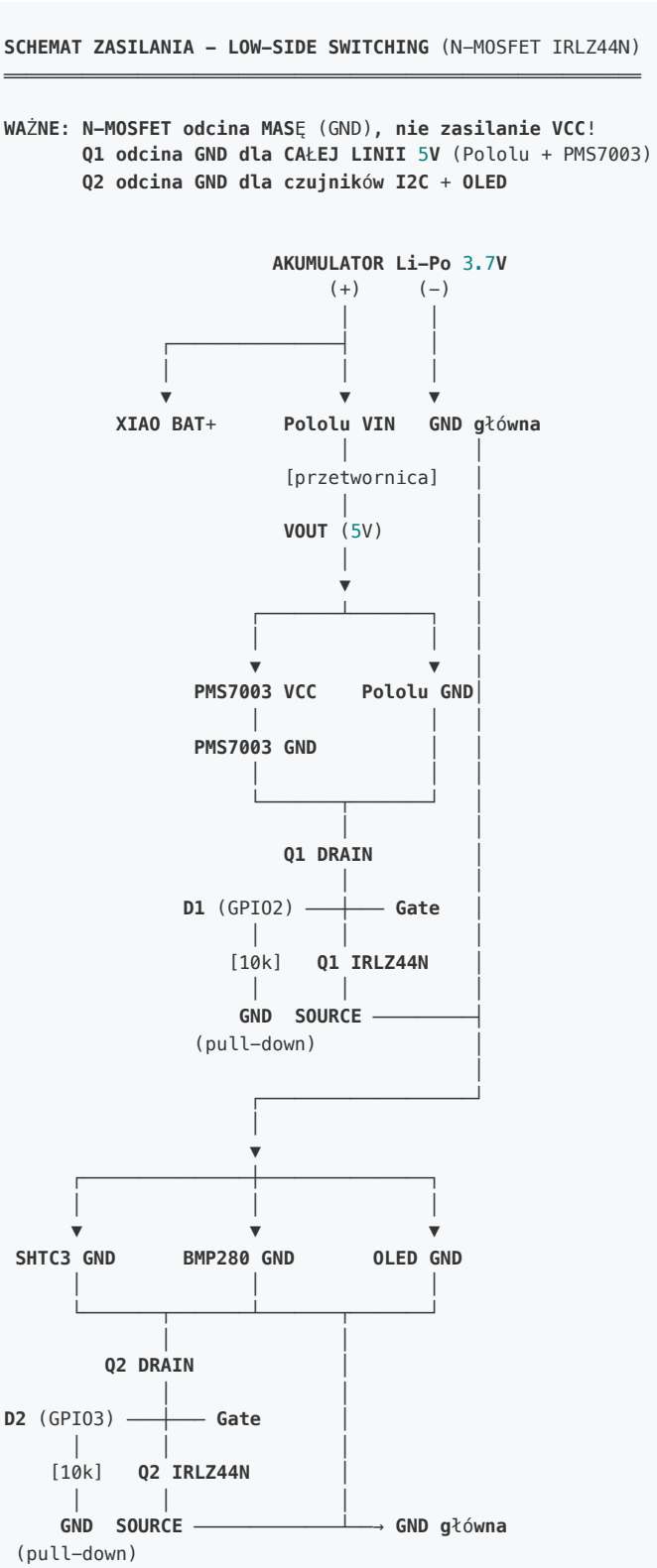


BUZZER (bezpośrednio do GPIO)
==========

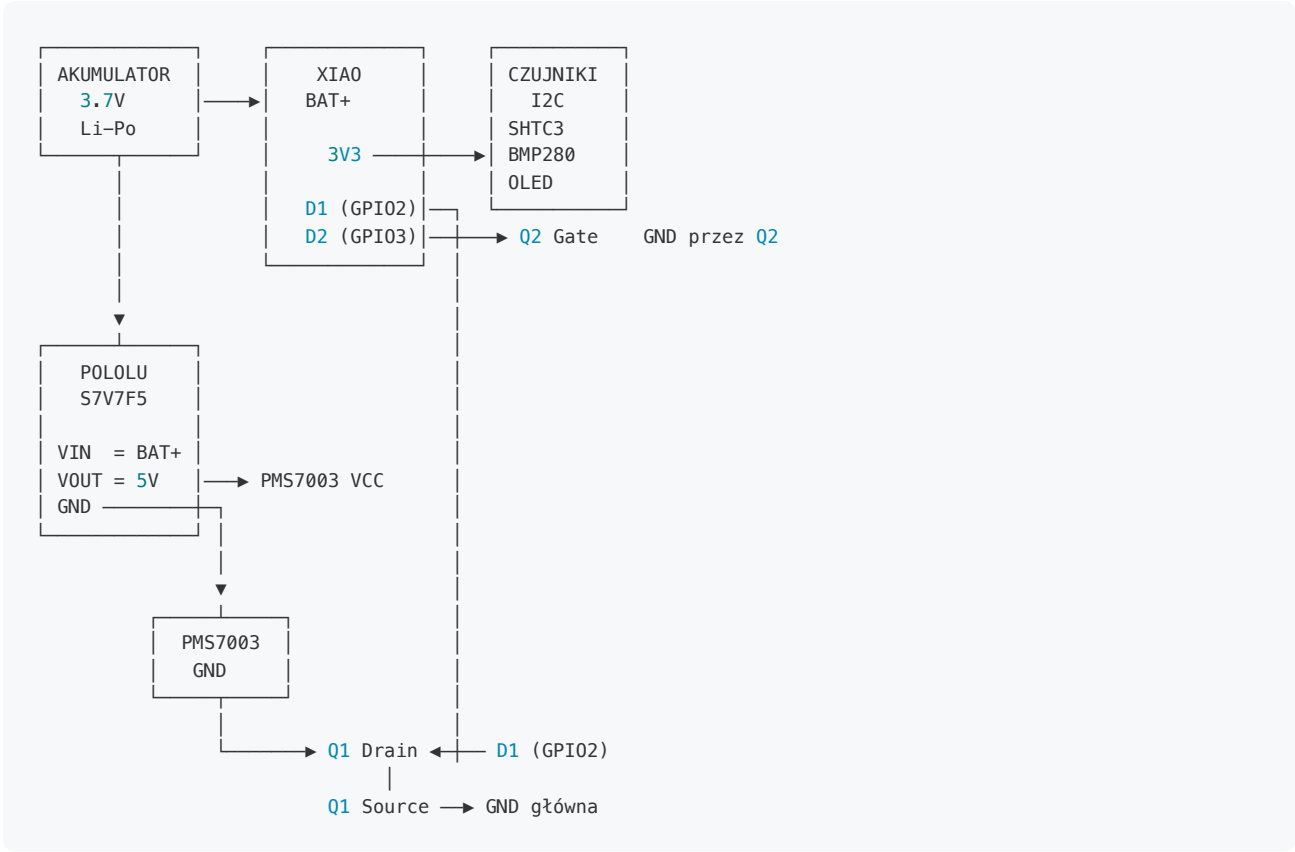


```
    D0 (GPIO1) ──> Buzzer (S)
    GND główna ──> Buzzer (−)
```

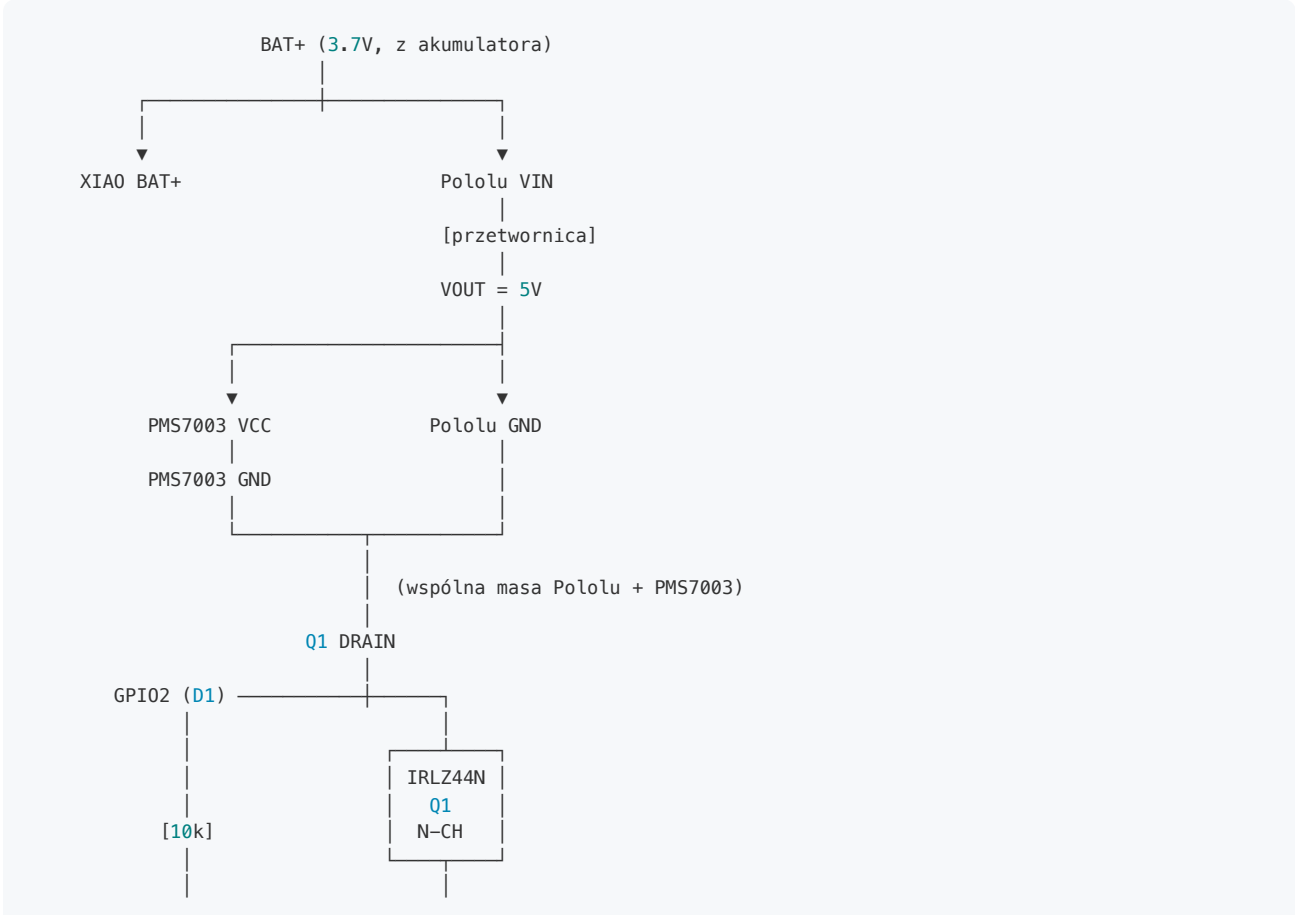
Schemat Zasilania z MOSFETami N-channel (LOW-SIDE SWITCHING)



Schemat Blokowy Zasilania



Szczegółowy Schemat Q1 (Linia 5V)

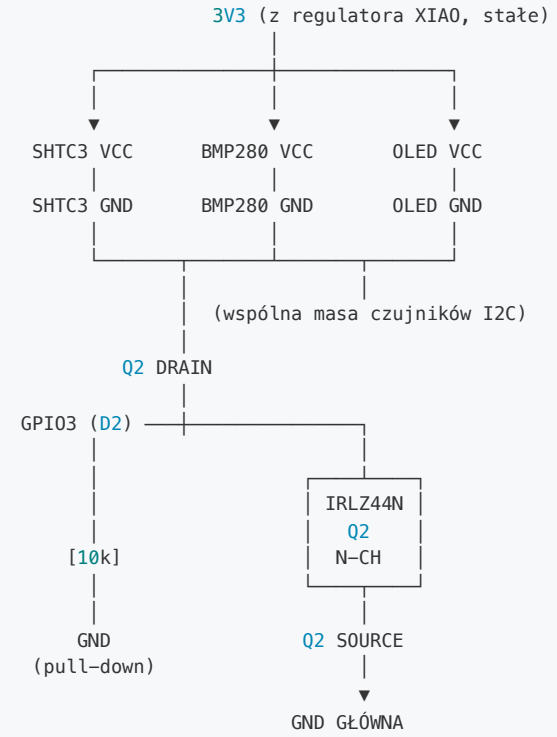


```
            GND                Q1 SOURCE
        (pull-down)                 │
                                    ▼
                              GND GŁÓWNA
                          (masa akumulatora)


DZIAŁANIE:
  GPIO2 = HIGH (3.3V) ──► Q1 ON  ──► GND połączone ──► Pololu+PMS WŁĄCZONE
  GPIO2 = LOW  (0V)   ──► Q1 OFF ──► GND odcięte   ──► Pololu+PMS WYŁĄCZONE

KLUCZOWA ZMIANA: GND Pololu idzie przez Q1 (nie bezpośrednio do masy!)
                 Dzięki temu wyłączenie Q1 odcina prąd jałowy (~2mA)
```

Szczegółowy Schemat Q2 (Linia 3.3V - I2C)




```
DZIAŁANIE:
  GPIO3 = HIGH (3.3V) ──► Q2 ON  ──► GND połączone ──► Czujniki WŁĄCZONE
  GPIO3 = LOW  (0V)   ──► Q2 OFF ──► GND odcięte   ──► Czujniki WYŁĄCZONE
```

Logika Sterowania

```
┌──────────────────────────────────────────────────────────────┐
│  STEROWANIE N-MOSFET (LOW-SIDE)                              │
├──────────────────────────────────────────────────────────────┤
│  GPIO = HIGH (3.3V) ──► MOSFET ON  ──► GND podłączone ──► ON │
│  GPIO = LOW  (0V)   ──► MOSFET OFF ──► GND odcięte    ──► OFF│
└──────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────┐
│  STANY PRZY STARCIE/RESECIE ESP32                            │
├──────────────────────────────────────────────────────────────┤
│  GPIO w stanie Hi-Z (wysoka impedancja)                      │
│  Rezystor 10k pull-down ściąga Gate do GND                   │
```

```
│  MOSFET OFF ──► peryferia WYŁĄCZONE ──► ochrona baterii          │
└──────────────────────────────────────────────────────────────────┘
```

Podsumowanie Połączeń Zasilania

```
AKUMULATOR 3.7V
    │
    ├──────────► XIAO BAT+ (zasilanie mikrokontrolera)
    │
    ├──────────► Pololu VIN (wejście przetwornicy)
    │
    └──────────► GND główna
                    │
                    ├──────────► XIAO GND
                    │
                    ├──────────► Q1 Source
                    │
                    ├──────────► Q2 Source
                    │
                    └──────────► Buzzer (−)

Q1 Drain ◄──────── Pololu GND + PMS7003 GND (wspólnie!)

Q2 Drain ◄──────── SHTC3 GND + BMP280 GND + OLED GND (wspólnie!)
```

Schemat Pojedynczego Switcha N-MOSFET (Low-Side)

```
ZASADA DZIAŁANIA N-MOSFET LOW-SIDE SWITCHING
═══════════════════════════════════════════

          VCC (3.3V lub 5V - stałe zasilanie)
           │
           │
      ┌────┴─────┐
      │URZĄDZENIE│  (czujnik, przetwornica, itp.)
      └────┬─────┘
           │
           │ GND urządzenia
           │
           ▼
      ┌──────────┐
      │          │
      │  DRAIN   │ ◄── tu podłączasz GND urządzenia
      │          │
      │ IRLZ44N  │
      │  N-CH    │
      │          │
      │  GATE ───┼──────┐
      │          │      │
      │ SOURCE   │    [10k] pull-down
      │          │      │
      └────┬─────┘     GND
           │
           │
           ▼
      GND GŁÓWNA (masa akumulatora)


STEROWANIE Z GPIO:

    GPIO (D1 lub D2)
           │
           └──────────► GATE
           │
```

```
          [10k]
            |
           GND


LOGIKA:
  GPIO = HIGH (3.3V)  ──►  MOSFET ON   ──►  GND połączone  ──►  DZIAŁA
  GPIO = LOW  (0V)    ──►  MOSFET OFF  ──►  GND odcięte    ──►  WYŁĄCZONE
  GPIO = Hi-Z (reset) ──►  pull-down   ──►  MOSFET OFF     ──►  WYŁĄCZONE


PINOUT IRLZ44N (TO-220, widok od przodu - strona z napisem):

        ┌──────────────┐
        │   IRLZ44N    │
        │              │
        │  ┌────────┐  │
        │  │        │  │
        │  │ metal  │  │  ◄── radiator (połączony z Drain)
        │  │        │  │
        │  └────────┘  │
        └──┬──┬──┬─────┘
           │  │  │
           │  │  │
           G  D  S
          (1)(2)(3)

    Pin 1 (G) - Gate   - sterowanie z GPIO + 10k do GND
    Pin 2 (D) - Drain  - GND urządzenia (urządzeń)
    Pin 3 (S) - Source - GND główna (masa akumulatora)


PARAMETRY IRLZ44N (logic-level MOSFET):
  • V_GS(th) = 1-2V      (włącza się przy 3.3V z GPIO - OK!)
  • R_DS(on) = 0.065 Ω   (bardzo niskie straty)
  • I_D = 47A            (dużo więcej niż potrzeba)
  • V_DS = 55V           (dużo więcej niż potrzeba)
  • Obudowa TO-220       (duża, łatwa do lutowania)
```

Schemat Sterowania Buzzerem (bezpośrednio do GPIO)

```
BUZZER AKTYWNY - PODŁĄCZENIE
═══════════════════════════


    3.3V (z XIAO, zasilanie stałe)
     │
     │ (+)
     ▼
    ┌┴┐
┌───┴─┴──┐
│ BUZZER │
│ aktywny│
│(zielony)│
└────┬───┘
     │
     │ (S) sygnał
     ▼
  D0 (GPIO1)  ◄── sterowanie ON/OFF

     │ (-)
     ▼
GND główna


PINY MODUŁU BUZZERA:
  (+) = VCC  ──► 3.3V z XIAO
  (-) = GND  ──► GND główna (bezpośrednio, nie przez MOSFET!)
  (S) = SIG  ──► D0 (GPIO1)
```

```
STEROWANIE:
  GPIO1 = HIGH ──► buzzer włączony (piszczy)
  GPIO1 = LOW  ──► buzzer wyłączony (cicho)

POBÓR PRĄDU: ~10–15mA (bezpiecznie dla GPIO ESP32, max 40mA)
```

Moduł z buzzerem aktywnym (zielony) - Botland 4526:

- Pobór prądu ~10-15mA - bezpiecznie w zakresie GPIO ESP32 (max 40mA)
- Piny: S (sygnał), - (GND), + (VCC 3.3V)
- GPIO HIGH = buzzer włączony, GPIO LOW = buzzer wyłączony
- **Nie wymaga tranzystora** - można podłączyć bezpośrednio

Zarządzanie Energią (Power Management)

Strategia Oszczędzania Energii

System wykorzystuje dwa MOSFETy N-channel (IRLZ44N) do odcinania masy peryferiów (low-side switching):

MOSFET	GPIO	Funkcja	Kontrolowane urządzenia	Logika
Q1	D1 (GPIO2)	PWR_5V_EN	GND dla Pololu + PMS7003	HIGH=ON
Q2	D2 (GPIO3)	PWR_3V3_EN	GND dla SHTC3, BMP280, OLED	HIGH=ON

Tryby Pracy Systemu

Tryb	Q1 (5V)	Q2 (3.3V)	ESP32	Opis
MEASUREMENT	ON	ON	Active	Pełny pomiar (PM + I2C)
DISPLAY	OFF	ON	Active	Wyświetlanie na OLED (przycisk)
DEEP_SLEEP	OFF	OFF	Deep Sleep	Minimalne zużycie

Adaptacyjny Interwał Pomiarów

System dostosowuje częstotliwość pomiarów do aktualnej jakości powietrza:

```
PROGI JAKOŚCI POWIETRZA (PM2.5)
==============================

PM2.5 [ug/m3]     Jakość           Interwał     Buzzer
------------------------------------------------------
0 – 15            DOBRA            co 2 godz    –
15 – 35           UMIARKOWANA      co 1 godz    –
35 – 75           NIEZDROWA        co 30 min    –
> 75              ZŁA              co 15 min    ALARM!
```

Synchronizacja Pomiarów

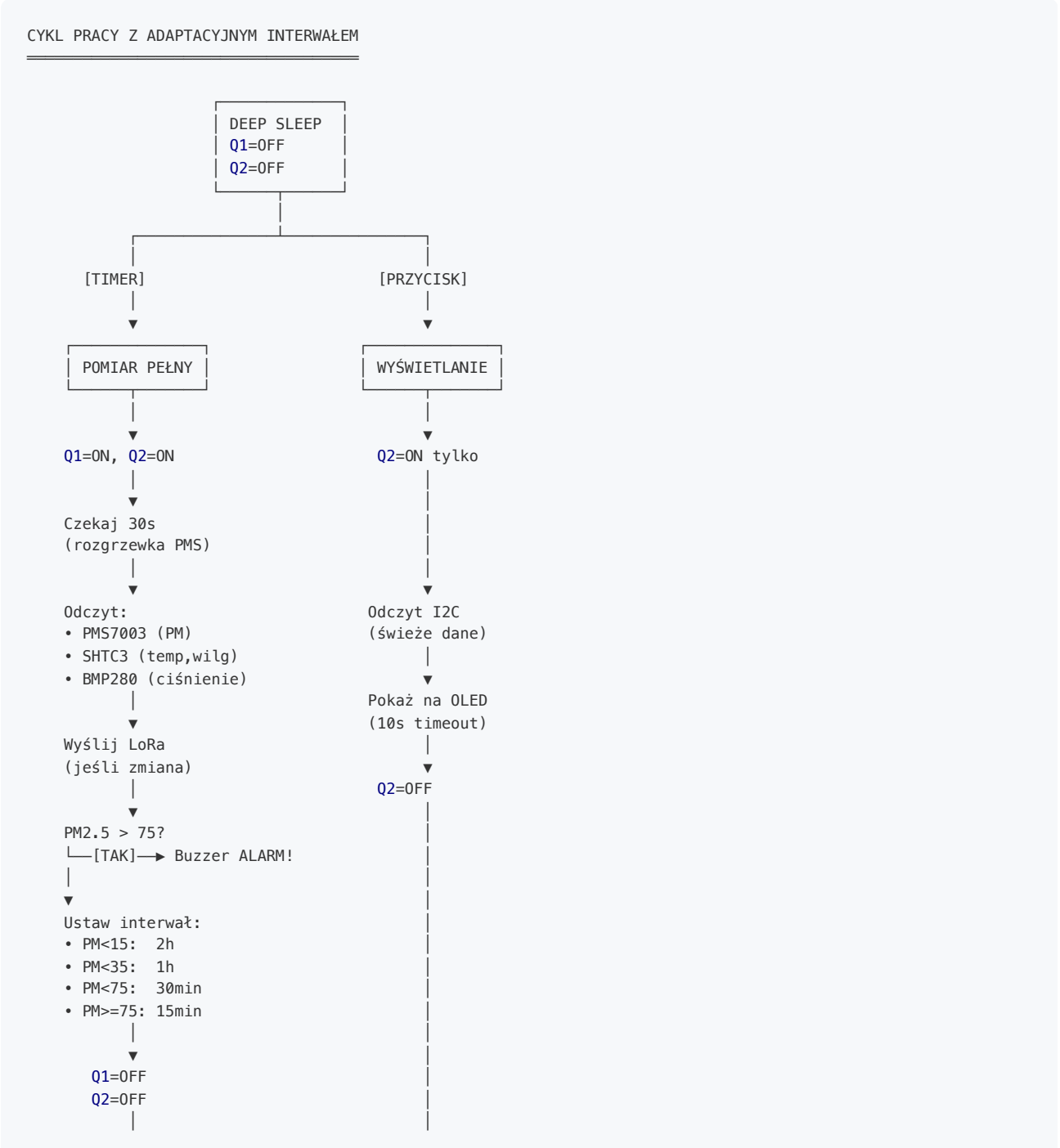
Wszystkie pomiary wykonywane razem - jedno wybudzenie = wszystkie odczyty:

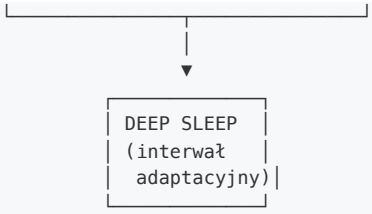
Jakość powietrza	Interwał	Pomiary na wybudzenie
Dobra (PM<15)	co 2 godz	PMS + I2C + LoRa
Umiarkowana	co 1 godz	PMS + I2C + LoRa
Niezdrowa	co 30 min	PMS + I2C + LoRa
Zła (PM>75)	co 15 min	PMS + I2C + LoRa + Buzzer

Wybudzenie przyciskiem - tylko wyświetlenie (bez PMS):

- Q2 ON -> Odczyt I2C -> Pokaż na OLED -> Q2 OFF -> Sleep

Algorytm Adaptacyjny - Schemat





Kod Adaptacyjnego Zarządzania Energią

```
// ============================================
// DEFINICJE PINÓW
// ============================================
#define PWR_5V_EN_PIN   2   // D1 - Q1 (Pololu + PMS7003)
#define PWR_3V3_EN_PIN  3   // D2 - Q2 (I2C + OLED)
#define BUZZER_PIN      1   // D0
#define BUTTON_PIN      4   // D3 - przycisk wybudzenia (do ustalenia)

// ============================================
// PROGI PM2.5 I INTERWAŁY (w mikrosekundach)
// ============================================
#define PM25_GOOD           15      // ug/m3
#define PM25_MODERATE       35      // ug/m3
#define PM25_UNHEALTHY      75      // ug/m3

#define INTERVAL_GOOD       (2ULL * 60 * 60 * 1000000)  // 2 godziny
#define INTERVAL_MODERATE   (1ULL * 60 * 60 * 1000000)  // 1 godzina
#define INTERVAL_UNHEALTHY  (30ULL * 60 * 1000000)      // 30 minut
#define INTERVAL_BAD        (15ULL * 60 * 1000000)      // 15 minut

// ============================================
// ZMIENNE GLOBALNE (przechowywane w RTC memory)
// ============================================
RTC_DATA_ATTR uint16_t lastPM25 = 0;
RTC_DATA_ATTR float lastTemp = 0;
RTC_DATA_ATTR float lastHum = 0;
RTC_DATA_ATTR float lastPressure = 0;

// ============================================
// FUNKCJE STEROWANIA ZASILANIEM
// ============================================
void setupPowerControl() {
    pinMode(PWR_5V_EN_PIN, OUTPUT);
    pinMode(PWR_3V3_EN_PIN, OUTPUT);
    pinMode(BUZZER_PIN, OUTPUT);

    digitalWrite(PWR_5V_EN_PIN, LOW);   // Q1 OFF
    digitalWrite(PWR_3V3_EN_PIN, LOW);  // Q2 OFF
    digitalWrite(BUZZER_PIN, LOW);
}

void enablePMS() {
    digitalWrite(PWR_5V_EN_PIN, HIGH);
    delay(100);
}

void disablePMS() {
    digitalWrite(PWR_5V_EN_PIN, LOW);
}

void enableI2C_OLED() {
    digitalWrite(PWR_3V3_EN_PIN, HIGH);
    delay(50);
}

void disableI2C_OLED() {
    digitalWrite(PWR_3V3_EN_PIN, LOW);
```

```
}

void alarmBuzzer() {
    for(int i = 0; i < 3; i++) {
        digitalWrite(BUZZER_PIN, HIGH);
        delay(200);
        digitalWrite(BUZZER_PIN, LOW);
        delay(200);
    }
}

// ============================================
// FUNKCJA ADAPTACYJNEGO INTERWAŁU
// ============================================
uint64_t getAdaptiveInterval(uint16_t pm25) {
    if (pm25 < PM25_GOOD) {
        return INTERVAL_GOOD;        // 2h - powietrze czyste
    } else if (pm25 < PM25_MODERATE) {
        return INTERVAL_MODERATE;    // 1h - umiarkowane
    } else if (pm25 < PM25_UNHEALTHY) {
        return INTERVAL_UNHEALTHY;   // 30min - niezdrowe
    } else {
        return INTERVAL_BAD;         // 15min - złe
    }
}

// ============================================
// GŁÓWNY CYKL POMIAROWY
// ============================================
void performMeasurementCycle() {
    // Włącz wszystkie peryferia
    enableI2C_OLED();
    enablePMS();

    // Czekaj na rozgrzewkę PMS7003 (30s)
    delay(30000);

    // Odczyty
    uint16_t pm25 = readPMS7003_PM25();
    float temp = readSHTC3_Temperature();
    float hum = readSHTC3_Humidity();
    float pressure = readBMP280_Pressure();

    // Wyłącz PMS (największy pobór prądu)
    disablePMS();

    // Alarm jeśli powietrze złe
    if (pm25 > PM25_UNHEALTHY) {
        alarmBuzzer();
    }

    // Wyślij przez LoRa (jeśli zmiana > 10%)
    if (hasSignificantChange(pm25, temp, hum, pressure)) {
        sendLoRaData(pm25, temp, hum, pressure);
    }

    // Zapisz wartości
    lastPM25 = pm25;
    lastTemp = temp;
    lastHum = hum;
    lastPressure = pressure;

    // Wyłącz peryferia
    disableI2C_OLED();

    // Oblicz następny interwał i idź spać
    uint64_t sleepTime = getAdaptiveInterval(pm25);
    esp_sleep_enable_timer_wakeup(sleepTime);
    esp_deep_sleep_start();
}
```

```
// ============================================
// WYBUDZENIE PRZYCISKIEM - TYLKO WYŚWIETLANIE
// ============================================
void displayOnButtonPress() {
    enableI2C_OLED();

    // Odczytaj świeże dane I2C (szybkie)
    float temp = readSHTC3_Temperature();
    float hum = readSHTC3_Humidity();
    float pressure = readBMP280_Pressure();

    // Pokaż na OLED (ostatni PM + świeże I2C)
    displayData(lastPM25, temp, hum, pressure);

    // Czekaj 10s lub do puszczenia przycisku
    delay(10000);

    disableI2C_OLED();
}

// ============================================
// SETUP I LOOP
// ============================================
void setup() {
    setupPowerControl();

    // Sprawdź powód wybudzenia
    esp_sleep_wakeup_cause_t wakeup_reason = esp_sleep_get_wakeup_cause();

    if (wakeup_reason == ESP_SLEEP_WAKEUP_EXT0) {
        // Wybudzenie przyciskiem
        displayOnButtonPress();
    } else {
        // Wybudzenie timerem lub pierwszy start
        performMeasurementCycle();
    }

    // Konfiguracja wybudzenia przyciskiem
    esp_sleep_enable_ext0_wakeup(GPIO_NUM_4, LOW);  // Przycisk na GPIO4

    // Powrót do sleep (interwał już ustawiony w performMeasurementCycle)
    esp_deep_sleep_start();
}

void loop() {
    // Nie używane - cała logika w setup()
}
```

Pobór Prądu - Szczegółowa Analiza

Komponent	Tryb aktywny	Odcięty (Q OFF)
ESP32-S3 active	25mA	-
ESP32-S3 deep sleep	14µA	-
PMS7003 + Pololu	102mA	**0mA**
SHTC3 + BMP280	1mA	**0mA**
OLED	20mA	**0mA**
SX1262 TX	120mA	1µA

Zużycie Energii - Jeden Cykl Pomiarowy

```
CYKL POMIAROWY (PMS + I2C + LoRa):
==================================
1. Rozgrzewka PMS (30s):    30s × 148mA = 1.23 mAh
2. Pomiar + przetwarzanie:   3s × 148mA = 0.12 mAh
3. Transmisja LoRa:          1s × 145mA = 0.04 mAh
4. Wyłączanie:               1s ×  25mA = 0.01 mAh
                             ----------------------
                       SUMA NA CYKL: ~1.4 mAh
```

Szacowany Czas Pracy Baterii 4000mAh

```
SCENARIUSZ: Adaptacyjny interwał
================================

Założenia:
- Bateria: 4000mAh
- Jeden cykl: ~1.4 mAh
- Deep sleep: 14µA (~0.014 mAh/h)
- Wyświetlanie na przycisk: 2x dziennie po 10s (~0.03 mAh/dzień)

POWIETRZE CZYSTE (PM < 15 ug/m3):
---------------------------------
Interwał: 2h = 12 cykli/dzień
Zużycie: 12 × 1.4 + 24 × 0.014 + 0.03 = 17.2 mAh/dzień
Czas pracy: 4000 / 17.2 = 232 dni = ~7.7 miesiąca

POWIETRZE UMIARKOWANE (PM 15-35 ug/m3):
---------------------------------------
Interwał: 1h = 24 cykle/dzień
Zużycie: 24 × 1.4 + 24 × 0.014 + 0.03 = 34.0 mAh/dzień
Czas pracy: 4000 / 34.0 = 118 dni = ~4 miesiące

POWIETRZE NIEZDROWE (PM 35-75 ug/m3):
-------------------------------------
Interwał: 30min = 48 cykli/dzień
Zużycie: 48 × 1.4 + 24 × 0.014 + 0.03 = 67.5 mAh/dzień
Czas pracy: 4000 / 67.5 = 59 dni = ~2 miesiące

POWIETRZE ZŁE (PM > 75 ug/m3):
------------------------------
Interwał: 15min = 96 cykli/dzień
Zużycie: 96 × 1.4 + 24 × 0.014 + 0.03 = 134.7 mAh/dzień
Czas pracy: 4000 / 134.7 = 30 dni = ~1 miesiąc
```

Podsumowanie Czasu Pracy Baterii

Jakość powietrza	Interwał	Czas pracy baterii
Dobra (PM<15)	co 2h	**~8 miesięcy**
Umiarkowana (PM 15-35)	co 1h	**~4 miesiące**
Niezdrowa (PM 35-75)	co 30min	**~2 miesiące**
Zła (PM>75)	co 15min	**~1 miesiąc**

> **UWAGA:** W typowych warunkach domowych (czyste/umiarkowane powietrze) urządzenie powinno pracować **4-8 miesięcy na jednym ładowaniu**.

Adresy I2C

Urządzenie	Adres domyślny	Adres alternatywny
SHTC3	0x70	- (stały)
BMP280	0x77	0x76 (SDO->GND)
OLED SSD1306	0x3C	0x3D (zależy od modułu)

Skan I2C - kod do weryfikacji adresów znajduje się w sekcji oprogramowania.

Połączenia PMS7003 (Adapter IDC-microUSB z CH340G)

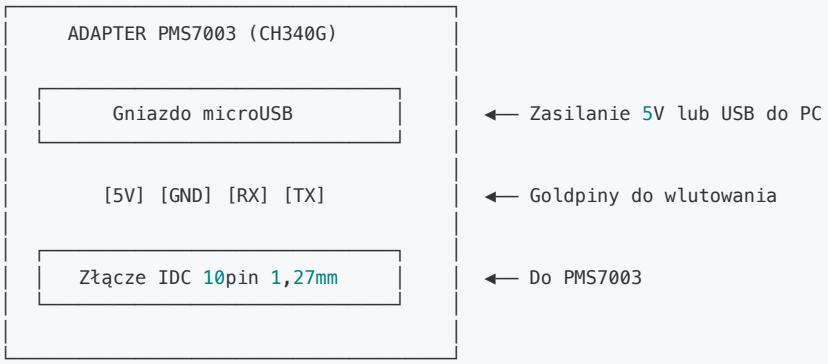
Opis Adaptera

Adapter IDC 10pin 1,27mm - microUSB dla czujnika PMS7003 (Botland DNG-13445):

- Konwertuje złącze IDC 10pin czujnika PMS7003 na wygodne piny i microUSB
- Wbudowany układ **CH340G** (konwerter UART<->USB)
- Gniazdo **microUSB** - może służyć do zasilania 5V lub komunikacji z PC
- Wyprowadzenia goldpin: **5V, GND, RX, TX**

Link: https://botland.com.pl/czujniki-czystosci-powietrza/13445-adapter-idc-10pin-127mm-microusb-dla-czujnika-pms7003-5904422372293.html

Pinout Adaptera PMS7003 (goldpiny)



Połączenia Adaptera z XIAO (przez goldpiny)

Pin Adaptera	Funkcja	Podłączyć do	Uwagi
5V	Zasilanie 5V	Pololu VOUT (5V)	Zasilanie czujnika
GND	Masa	**Q1 Drain**	Przez MOSFET! (low-side)
RX	UART RX (wejście adaptera)	XIAO D6 (GPIO43/TX)	TX XIAO -> RX adaptera

Pin Adaptera	Funkcja	Podłączyć do	Uwagi
TX	UART TX (wyjście adaptera)	XIAO D7 (GPIO44/RX)	TX adaptera -> RX XIAO

WAŻNE: Nie używać microUSB do zasilania!

> **W tym projekcie NIE używamy gniazda microUSB adaptera!**
>
> Zasilanie 5V dostarczamy przez pin **5V** z przetwornicy Pololu. Masę (GND) prowadzimy przez MOSFET Q1 (low-side switching).
>
> Gniazdo microUSB można użyć tylko do testowania z komputerem PC (wtedy adapter działa jako konwerter USB-UART).

Stary pinout IDC (dla referencji)

```
Oryginalny connector IDC 10pin PMS7003 (1.27mm):
┌─────────────────────┐
│ 1  2  3  4  5       │
│ 6  7  8  9  10      │
└─────────────────────┘

Pin 1: VCC (5V)
Pin 2: GND
Pin 3: SET (sterowanie sleep, HIGH=praca, LOW=sleep)
Pin 4: RX (wejście UART, 3.3V TTL)
Pin 5: TX (wyjście UART, 3.3V TTL)
Pin 6: RESET (aktywny LOW)
Pin 7–10: NC/GND

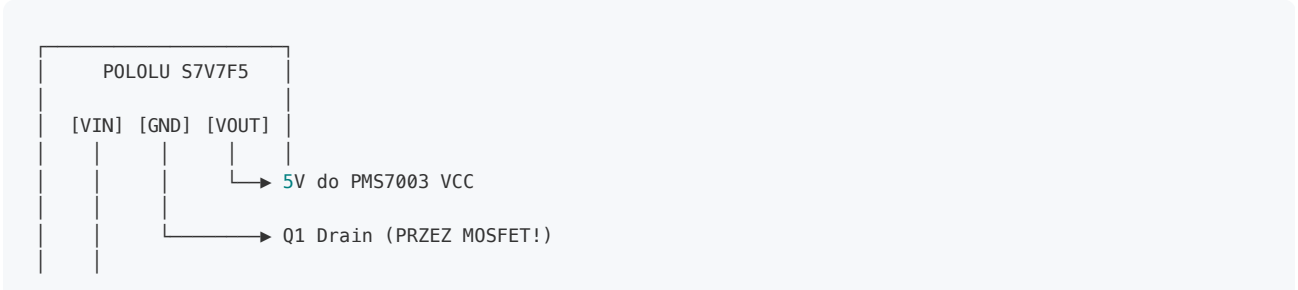
Adapter wyprowadza tylko: 5V, GND, RX, TX (piny SET i RESET niedostępne)
```

Połączenia Przetwornicy Pololu S7V7F5

Pololu Pin	Funkcja	Podłączony do	Uwagi
VIN	Wejście 2.7-11.8V	Akumulator BAT+ (3.7V)	Bezpośrednio do baterii
GND	Masa	**Q1 Drain**	**PRZEZ MOSFET!**
VOUT	Wyjście 5V	PMS7003 VCC (adapter 5V pin)	Zasilanie czujnika

> **WAŻNA ZMIANA:** GND Pololu idzie przez Q1 (nie bezpośrednio do masy!) Dzięki temu wyłączenie Q1 całkowicie odcina przetwornicę i eliminuje pobór prądu jałowego (~2mA).

Pinout Pololu S7V7F5:



```
│      └──────────────► BAT+ (bezpośrednio)
└──────────────────────┘
```

Lista Połączeń (Instrukcja Montażu)

1. ZASILANIE GŁÓWNE

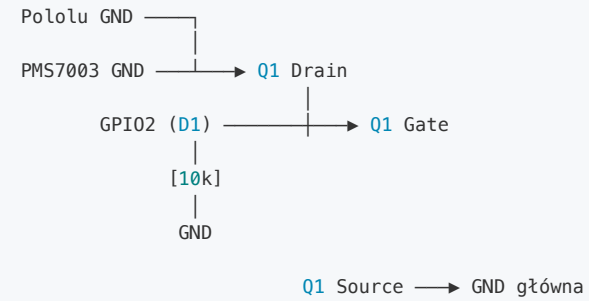
Z	Do	Uwagi
Akumulator (+)	XIAO BAT+	Zasilanie główne
Akumulator (-)	GND główna	Masa główna
Akumulator (+)	Pololu VIN	Zasilanie przetwornicy

> **UWAGA:** GND Pololu NIE idzie bezpośrednio do masy - idzie przez Q1!

2. MOSFET Q1 (IRLZ44N) - sterowanie linii 5V (Pololu + PMS7003)

Pin IRLZ44N	Podłączyć do	Uwagi
G (Gate)	XIAO D1 (GPIO2)	Sterowanie
G (Gate)	Rezystor 10k -> GND	Pull-down (domyślnie OFF)
D (Drain)	Pololu GND + PMS7003 GND	**WSPÓLNIE!**
S (Source)	GND główna	Masa systemu

Schemat połączeń Q1:



3. MOSFET Q2 (IRLZ44N) - sterowanie czujników I2C

Pin IRLZ44N	Podłączyć do	Uwagi
G (Gate)	XIAO D2 (GPIO3)	Sterowanie
G (Gate)	Rezystor 10k -> GND	Pull-down (domyślnie OFF)
D (Drain)	GND: SHTC3, BMP280, OLED	Wspólna masa czujników
S (Source)	GND główna	Masa systemu



4. PRZETWORNICA POLOLU S7V7F5

Pin Pololu	Podłączyć do	Uwagi
VIN	Akumulator BAT+	Bezpośrednio!
GND	**Q1 Drain**	**PRZEZ MOSFET!**
VOUT	PMS7003 VCC (adapter pin 5V)	Zasilanie 5V

5. PMS7003 (czujnik pyłu) - przez adapter IDC-microUSB

Pin Adaptera	Podłączyć do	Uwagi
5V	Pololu VOUT	Zasilanie 5V
GND	**Q1 Drain**	**PRZEZ MOSFET!**
RX	XIAO D6 (GPIO43/TX)	UART TX->RX
TX	XIAO D7 (GPIO44/RX)	UART RX<-TX

6. MAGISTRALA I2C (wspólna dla 3 urządzeń)

Sygnał	XIAO Pin	SHTC3	BMP280	OLED
SDA	D4 (GPIO5)	SDA	SDA	SDA
SCL	D5 (GPIO6)	SCL	SCL	SCL
VCC	3V3	VCC	VCC	VCC
GND	**Q2 Drain**	GND	GND	GND

7. BUZZER

Pin Buzzer	Podłączyć do	Uwagi
S (Signal)	XIAO D0 (GPIO1)	Sterowanie
- (GND)	GND główna	Masa (bezpośrednio!)

Pin Buzzer	Podłączyć do	Uwagi
+ (VCC)	XIAO 3V3	Zasilanie

8. PRZYCISK WYBUDZENIA

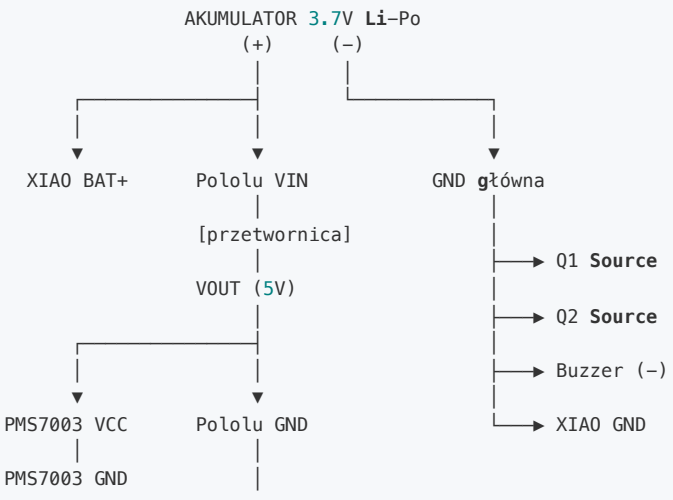
Przycisk	Podłączyć do	Uwagi
Pin 1	XIAO D3 (GPIO4)	Wbudowany pull-up ESP32
Pin 2	GND główna	Przycisk NO (normalnie otwarty)

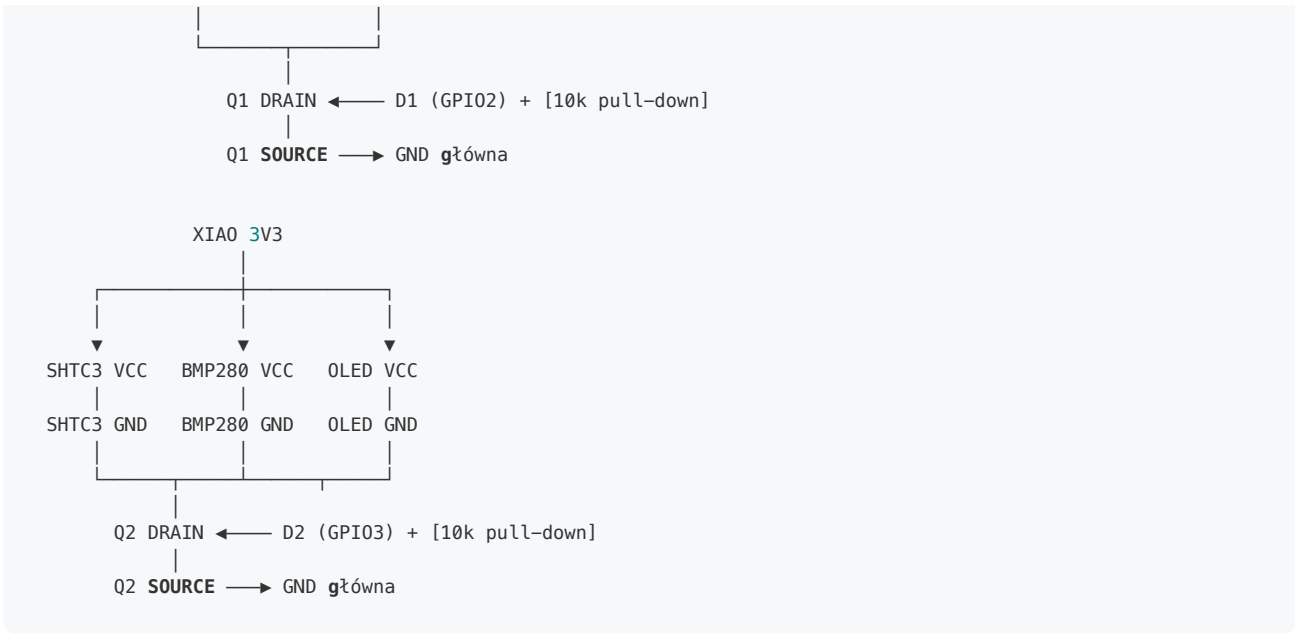
Działanie: Naciśnięcie przycisku łączy GPIO4 z GND -> wybudzenie z deep sleep -> wyświetlenie danych na OLED.

9. PODSUMOWANIE PINÓW XIAO

Pin XIAO	GPIO	Funkcja	Podłączone do
D0	GPIO1	BUZZER	Buzzer S
D1	GPIO2	PWR_5V_EN	Q1 Gate + 10k->GND
D2	GPIO3	PWR_3V3_EN	Q2 Gate + 10k->GND
D3	GPIO4	BUTTON_WAKE	Przycisk NO do GND
D4	GPIO5	I2C SDA	SHTC3, BMP280, OLED
D5	GPIO6	I2C SCL	SHTC3, BMP280, OLED
D6	GPIO43	UART TX	PMS7003 RX
D7	GPIO44	UART RX	PMS7003 TX
3V3	-	3.3V	Czujniki I2C VCC, Buzzer +
GND	-	GND	Q1 Source, Q2 Source, Buzzer -, Przycisk
BAT+	-	Bateria +	Akumulator +, Pololu VIN

10. DIAGRAM POŁĄCZEŃ - WIDOK CAŁOŚCIOWY





11. CHECKLIST PRZED WŁĄCZENIEM

- [ ] Brak zwarć między VCC a GND (multimetr - tryb ciągłości)
- [ ] Q1 Gate ma rezystor 10k do GND
- [ ] Q2 Gate ma rezystor 10k do GND
- [ ] Pololu VIN podłączony do BAT+ (bezpośrednio)
- [ ] **Pololu GND podłączony do Q1 Drain (NIE do GND głównej!)**
- [ ] **PMS7003 GND podłączony do Q1 Drain (NIE do GND głównej!)**
- [ ] Czujniki I2C GND do Q2 Drain (nie do GND głównej!)
- [ ] Q1 Source i Q2 Source do GND głównej
- [ ] Buzzer (-) bezpośrednio do GND głównej
- [ ] Przycisk podłączony między D3 (GPIO4) a GND
- [ ] Antena LoRa podłączona do SX1262

Wymagane Biblioteki Arduino

```
// Komunikacja LoRa
#include <RadioLib.h>

// Czujniki I2C
#include <Wire.h>
#include <Adafruit_SHTC3.h>        // lub SensirionI2CShtc3
#include <Adafruit_BMP280.h>

// Wyświetlacz OLED
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

// Czujnik pyłu (UART)
// Biblioteka PMS lub ręczna obsługa protokołu
```

Fazy Projektu

Faza 1: Koncepcja i Planowanie (AKTUALNA) ✓

Status: Zakończona

- [x] Lista komponentów hardware
- [x] Analiza kompatybilności napięć
- [x] Schemat połączeń
- [x] Analiza poboru prądu

Faza 2: Montaż Sprzętu

Cel: Połączenie wszystkich elementów na płytce uniwersalnej.

Zadania:

- [ ] Przylutowanie goldpinów do XIAO
- [ ] Montaż magistrali I2C (3.3V, GND, SDA, SCL)
- [ ] Podłączenie czujników I2C
- [ ] Podłączenie PMS7003 (UART + zasilanie 5V)
- [ ] Podłączenie OLED
- [ ] Podłączenie buzzera (bezpośrednio do GPIO)
- [ ] Podłączenie akumulatora
- [ ] Test ciągłości połączeń (multimetr)

Faza 3: Implementacja Oprogramowania

Baza kodu: `air_condition.ino` - działająca komunikacja SX1262

Zadania:

- [ ] Test skanera I2C
- [ ] Odczyt SHTC3 (temperatura, wilgotność)
- [ ] Odczyt BMP280 (ciśnienie, temperatura)
- [ ] Odczyt PMS7003 (PM1.0, PM2.5, PM10)
- [ ] Wyświetlanie danych na OLED
- [ ] Integracja z komunikacją LoRa
- [ ] Alarmy (buzzer przy przekroczeniu progów)
- [ ] Tryby oszczędzania energii

Faza 4: Integracja i Testy

Zadania:

- [ ] Kalibracja czujników
- [ ] Testy zasięgu LoRa
- [ ] Optymalizacja zużycia energii
- [ ] Testy długotrwałe

Specyfikacja Techniczna

Parametry LoRa (aktualna konfiguracja)

Parametr	Wartość
Częstotliwość	868.0 MHz
Szerokość pasma (BW)	125.0 kHz
Spreading Factor (SF)	7
Coding Rate (CR)	5 (4/5)
Sync Word	0x34
Moc nadawania	14 dBm
Preambuła	8 symboli

Zakresy Pomiarowe Czujników

Czujnik	Parametr	Zakres	Dokładność
SHTC3	Temperatura	-40°C do +125°C	±0.2°C
SHTC3	Wilgotność	0-100% RH	±2% RH
BMP280	Ciśnienie	300-1100 hPa	±1 hPa
BMP280	Temperatura	-40°C do +85°C	±1°C
PMS7003	PM1.0/2.5/10	0-500 µg/m³	±10%

Uwagi i Ograniczenia

1. **PMS7003 - rozgrzewka**: Czujnik wymaga ~30 sekund rozgrzewki przed wiarygodnym pomiarem
2. **PMS7003 - żywotność**: Wentylator ma ograniczoną żywotność (~8000h), nie trzymać włączonego ciągle
3. **PMS7003 - zasilanie**: VCC z Pololu (5V), GND przez MOSFET Q1 (low-side switching)
4. **I2C pull-up**: Moduły DFRobot mają wbudowane rezystory pull-up, nie trzeba dodawać zewnętrznych
5. **Bateria**: XIAO ma wbudowany układ ładowania Li-Po (100mA), ładowanie przez USB-C
6. **Antena LoRa**: Niezbędna do prawidłowej pracy SX1262 - podłączyć przed włączeniem!
7. **Pololu S7V7F5**: VIN do BAT+ (bezpośrednio), **GND przez Q1** - wyłączenie Q1 całkowicie odcina przetwornicę
8. **MOSFETy IRLZ44N - logika**: N-channel low-side: **GPIO HIGH = urządzenie ON**, GPIO LOW = OFF
9. **Pull-down rezystory (10k)**: Zapewniają domyślnie WYŁĄCZONE peryferia przy starcie/resecie ESP32
10. **Deep Sleep**: Przy wybudzeniu GPIO wracają do stanu domyślnego - pull-down utrzymuje OFF
11. **"Pływająca masa"**: Gdy MOSFET OFF, peryferia mają VCC ale nie GND - bezpieczne, nie działa
12. **IRLZ44N obudowa TO-220**: Duża (wymaga miejsca), ale łatwa do lutowania na płytce uniwersalnej
13. **IRLZ44N parametry**: V_GS(th)=1-2V, R_DS(on)=0.065ohm, I_D=47A - nadmiarowe, ale pewne
14. **Eliminacja prądu jałowego Pololu**: GND Pololu przez Q1 = 0mA w trybie sleep (zamiast ~2mA)
15. **Przycisk wybudzenia (GPIO4)**: Używa wbudowanego pull-up ESP32, normalnie otwarty do GND
16. **Adaptacyjny interwał**: Częstotliwość pomiarów zależy od jakości powietrza (2h-15min)

Następne Kroki

1. **Zakup brakujących elementów:**
   - Pololu S7V7F5 - przetwornica 5V
   - Rezystory 10kΩ (2 szt) - pull-down dla bramek MOSFET
   - Przycisk tact switch - wybudzenie z deep sleep
   - ~~IRLZ44N N-MOSFET~~ - **KUPIONY** (5 szt)
2. **Przygotowanie płytki uniwersalnej** - zaplanowanie układu z MOSFETami TO-220
3. **Lutowanie i montaż** - zgodnie ze schematem low-side switching
4. **Testy połączeń** - przed podłączeniem zasilania sprawdzić multimetrem
5. **Upload kodu testowego** - skan I2C, test power switching

Historia Zmian

Data	Wersja	Opis
2025-12-06	0.1	Utworzenie dokumentu, wstępna specyfikacja
2025-12-06	0.2	Dodanie listy hardware, schematu połączeń, analizy kompatybilności i poboru prądu
2025-12-06	0.3	Dodanie przetwornicy Pololu S7V7F5 - rozwiązanie problemu zasilania 5V z baterii
2025-12-06	0.4	Dodanie MOSFETów AO3401 do sterowania zasilaniem peryferiów, sekcja Power Management
2025-12-06	0.5	**Zmiana MOSFET z AO3401 (P-ch) na IRLZ44N (N-ch logic-level)** - low-side switching, odwrócona logika sterowania (HIGH=ON), zaktualizowane schematy i kod
2025-12-06	0.6	Dodanie sekcji "Lista Połączeń (Instrukcja Montażu)" z checklistą przed włączeniem
2025-12-06	0.7	Aktualizacja o adapter PMS7003 IDC-microUSB (CH340G) - pinout adaptera (5V/GND/RX/TX), połączenia przez goldpiny
2025-12-06	0.8	**Buzzer: zmiana na moduł aktywny (zielony)** - bezpośrednio do GPIO bez tranzystora, dodano link Botland
2025-12-06	0.9	Dodanie sekcji "Kontekst dla Kolejnej Sesji" z pełnym podsumowaniem stanu projektu
2025-12-30	1.0	**WAŻNA ZMIANA: GND Pololu przez Q1** - eliminacja prądu jałowego przetwornicy (~2mA), zaktualizowane wszystkie schematy ASCII, poprawiona czytelność diagramów
2025-12-30	1.1	**Adaptacyjny algorytm pomiarów** - interwał zależny od PM2.5 (2h/1h/30min/15min), przycisk wybudzenia (GPIO4), usunięcie zduplikowanej sekcji poboru prądu, zaktualizowana sekcja kontekstu
2025-12-30	1.2	**Przywrócenie znaków Unicode** w schematach ASCII dla lepszej czytelności (┌ ┐ └ ┘ ├ ─ ┤ ┬ ┴ ▶ ◀ ▼)

Kontekst dla Kolejnej Sesji (Session Handoff)

> **CEL:** Ta sekcja zawiera wszystkie informacje niezbędne do kontynuacji pracy w nowej sesji AI. Przeczytaj tę sekcję na początku kolejnej sesji, aby szybko wdrożyć się w projekt.

Status Projektu (2025-12-30)

Faza	Status	Uwagi
Faza 1: Koncepcja	**ZAKOŃCZONA**	Dokumentacja kompletna
Faza 2: Hardware	**W TOKU**	Czeka na zakup + montaż
Faza 3: Software	DO REALIZACJI	Baza: `air_condition.ino` (LoRa działa)
Faza 4: Testy	DO REALIZACJI	-

Co Jest Gotowe

1. **Dokumentacja techniczna** - PROJECT.md v1.2 zawiera:
   - Kompletną listę BOM z linkami Botland
   - Schematy połączeń (ASCII art)
   - Architekturę zasilania z low-side switching (IRLZ44N)
   - **Adaptacyjny algorytm pomiarów** (interwał zależny od PM2.5)
   - Analizę poboru prądu i żywotności baterii (4-8 miesięcy)
   - Instrukcję montażu z checklistą
2. **Kod bazowy** - `air_condition.ino`:
   - Działająca komunikacja LoRa (SX1262)
   - Konfiguracja pinów dla Wio-SX1262 + XIAO ESP32-S3
   - Gotowy do rozbudowy o czujniki
3. **Zakupione elementy:**
   - XIAO ESP32-S3 Sense + Wio-SX1262 (B2B)
   - SHTC3 (DFRobot SEN0414)
   - BMP280 (DFRobot SEN0372)
   - PMS7003 + adapter IDC-microUSB (CH340G)
   - OLED 0.96" 128x64 I2C
   - Akumulator Li-Po 4000mAh
   - Moduł buzzera aktywnego (zielony)
   - **IRLZ44N N-MOSFET** (5 szt)

Do Zakupu (PILNE)

Element	Ilość	Link	Cena ~
Pololu S7V7F5	1 szt	https://botland.com.pl/przetwornice-step-updown-pololu/1696-pololu-s7v7f5-przetwornica-step-upstep-down-5v-1a-5904422359898.html	~25 zł

Element	Ilość	Link	Cena ~
Rezystor 10k THT	2 szt	dowolny sklep	~1 zł
Przycisk tact switch	1 szt	dowolny sklep	~1 zł

Kluczowe Decyzje Projektowe

1. **Low-Side Switching z N-MOSFET (IRLZ44N):**
   - Q1 odcina GND dla **Pololu + PMS7003** (GPIO2, HIGH=ON)
   - Q2 odcina GND dla czujników I2C + OLED (GPIO3, HIGH=ON)
   - Pull-down 10k = domyślnie OFF przy starcie
2. **GND Pololu przez Q1 (WAŻNE!):**
   - Pololu VIN -> BAT+ (bezpośrednio)
   - Pololu GND -> Q1 Drain (PRZEZ MOSFET!)
   - Eliminuje ~2mA prądu jałowego przetwornicy w trybie sleep
3. **Adaptacyjny interwał pomiarów PM2.5:**
   - PM < 15 µg/m³ (dobra): pomiar co **2 godziny**
   - PM 15-35 µg/m³ (umiarkowana): pomiar co **1 godzinę**
   - PM 35-75 µg/m³ (niezdrowa): pomiar co **30 minut**
   - PM > 75 µg/m³ (zła): pomiar co **15 minut** + **ALARM BUZZER**
4. **Przycisk wybudzenia (GPIO4):**
   - Naciśnięcie wybudza ESP32 z deep sleep
   - Wyświetla aktualne dane na OLED (bez włączania PMS)
   - Po 10s automatycznie wraca do sleep
5. **Adapter PMS7003 (IDC-microUSB z CH340G):**
   - Używamy tylko goldpiny: 5V, GND, RX, TX
   - NIE używamy gniazda microUSB do zasilania
   - Piny SET/RESET niedostępne - sterowanie przez MOSFET
6. **Buzzer aktywny (zielony moduł):**
   - Bezpośrednio do GPIO (bez tranzystora)
   - Piny: S->GPIO1, (-)-> GND główna, (+)->3.3V

Szacowany Czas Pracy Baterii 4000mAh

Jakość powietrza	Interwał	Czas pracy
Dobra (PM<15)	co 2h	**~8 miesięcy**
Umiarkowana (PM 15-35)	co 1h	**~4 miesiące**

Jakość powietrza	Interwał	Czas pracy
Niezdrowa (PM 35-75)	co 30min	**~2 miesiące**
Zła (PM>75)	co 15min	**~1 miesiąc**

Pinout XIAO - Szybka Ściąga

```
| Pin  | GPIO  | Funkcja      | Podłączone do              |
|------|-------|--------------|----------------------------|
| D0   | GPIO1 | BUZZER       | Buzzer S (bezpośrednio)    |
| D1   | GPIO2 | PWR_5V_EN    | Q1 Gate + 10k->GND         |
| D2   | GPIO3 | PWR_3V3_EN   | Q2 Gate + 10k->GND         |
| D3   | GPIO4 | BUTTON_WAKE  | Przycisk NO do GND         |
| D4   | GPIO5 | I2C SDA      | SHTC3, BMP280, OLED        |
| D5   | GPIO6 | I2C SCL      | SHTC3, BMP280, OLED        |
| D6   | GPIO43| UART TX      | PMS7003 RX (adapter)       |
| D7   | GPIO44| UART RX      | PMS7003 TX (adapter)       |
```

Schemat Zasilania - Szybki Podgląd

```
AKUMULATOR 3.7V
    |
    ├──► XIAO BAT+
    |
    ├──► Pololu VIN (bezpośrednio!)
    |
    └──► GND główna ──┬──► XIAO GND
                      |
                      ├──► Q1 Source
                      |
                      ├──► Q2 Source
                      |
                      ├──► Buzzer (-)
                      |
                      └──► Przycisk (pin 2)

Q1 (GPIO2): Pololu GND + PMS7003 GND ──► Q1 Drain

Q2 (GPIO3): SHTC3 GND + BMP280 GND + OLED GND ──► Q2 Drain
```

Adresy I2C

Urządzenie	Adres
SHTC3	0x70
BMP280	0x77
OLED SSD1306	0x3C

Kolejne Kroki (w kolejności)

ETAP A: Zakup (przed montażem)

- [ ] Zamówić Pololu S7V7F5
- [ ] Zamówić rezystory 10k (2 szt, THT)

- [ ] Zamówić przycisk tact switch

ETAP B: Montaż Hardware

- [ ] Zaplanować układ na płytce uniwersalnej (MOSFETy TO-220 wymagają miejsca)
- [ ] Przylutować goldpiny do XIAO
- [ ] Zmontować sekcję zasilania (Pololu + Q1)
- [ ] **Pololu GND do Q1 Drain (NIE do masy głównej!)**
- [ ] Zmontować sekcję I2C (Q2 + czujniki + OLED)
- [ ] Podłączyć PMS7003 przez adapter (GND do Q1 Drain!)
- [ ] Podłączyć buzzer
- [ ] Podłączyć przycisk (D3/GPIO4 do GND)
- [ ] **TEST MULTIMETREM** przed włączeniem (checklist w sekcji 11)

ETAP C: Software - Testy Jednostkowe

- [ ] Skan I2C (sprawdzić adresy 0x70, 0x77, 0x3C)
- [ ] Test SHTC3 (temp + wilgotność)
- [ ] Test BMP280 (ciśnienie + temp)
- [ ] Test OLED (wyświetlanie tekstu)
- [ ] Test PMS7003 (odczyt PM2.5)
- [ ] Test buzzera (prosty beep)
- [ ] Test power switching (Q1, Q2)
- [ ] Test przycisku wybudzenia

ETAP D: Software - Integracja

- [ ] Połączyć wszystkie odczyty w jeden cykl
- [ ] Wyświetlanie na OLED
- [ ] Wysyłanie przez LoRa
- [ ] Alarmy (buzzer przy przekroczeniu progów PM2.5)
- [ ] Deep sleep + adaptacyjny interwał pomiarów
- [ ] Wybudzenie przyciskiem (tylko wyświetlanie)

Kod Startowy - Power Switching + Przycisk

```
#define PWR_5V_EN_PIN   2   // D1 – Q1 (Pololu + PMS7003)
#define PWR_3V3_EN_PIN  3   // D2 – Q2 (I2C + OLED)
#define BUZZER_PIN      1   // D0
#define BUTTON_PIN      4   // D3 – przycisk wybudzenia

// Progi PM2.5
#define PM25_GOOD       15
#define PM25_MODERATE   35
#define PM25_UNHEALTHY  75

// Interwały (mikrosekundy)
#define INTERVAL_GOOD       (2ULL * 60 * 60 * 1000000)  // 2h
#define INTERVAL_MODERATE   (1ULL * 60 * 60 * 1000000)  // 1h
#define INTERVAL_UNHEALTHY  (30ULL * 60 * 1000000)      // 30min
#define INTERVAL_BAD        (15ULL * 60 * 1000000)      // 15min

void setupPowerControl() {
    pinMode(PWR_5V_EN_PIN, OUTPUT);
```

```
    pinMode(PWR_3V3_EN_PIN, OUTPUT);
    pinMode(BUZZER_PIN, OUTPUT);

    digitalWrite(PWR_5V_EN_PIN, LOW);   // OFF
    digitalWrite(PWR_3V3_EN_PIN, LOW);  // OFF
    digitalWrite(BUZZER_PIN, LOW);      // OFF
}

uint64_t getAdaptiveInterval(uint16_t pm25) {
    if (pm25 < PM25_GOOD) return INTERVAL_GOOD;
    if (pm25 < PM25_MODERATE) return INTERVAL_MODERATE;
    if (pm25 < PM25_UNHEALTHY) return INTERVAL_UNHEALTHY;
    return INTERVAL_BAD;
}

void setupWakeupButton() {
    esp_sleep_enable_ext0_wakeup(GPIO_NUM_4, LOW);
}
```

Biblioteki Arduino (do zainstalowania)

```
- RadioLib (LoRa SX1262) - już używana
- Wire (I2C) - wbudowana
- Adafruit_SHTC3 lub SensirionI2CShtc3
- Adafruit_BMP280
- Adafruit_GFX + Adafruit_SSD1306 (OLED)
- PMS (opcjonalnie, można ręcznie parsować UART)
```

Znane Problemy / Uwagi

1. **PMS7003 rozgrzewka** - 30 sekund przed wiarygodnym pomiarem
2. **PMS7003 żywotność** - ~8000h wentylatora, nie włączać ciągle
3. **Antena LoRa** - ZAWSZE podłączyć przed włączeniem zasilania!
4. **IRLZ44N TO-220** - duże, zaplanować miejsce na płytce
5. **Adapter PMS7003** - goldpiny mogą wymagać wlutowania
6. **Pololu GND** - MUSI iść przez Q1, nie bezpośrednio do masy!
7. **Przycisk** - używa wbudowanego pull-up ESP32, normalnie otwarty

Pliki Projektu

```
/Users/marek.piotrowski/Projects/xiao/air_condition/
├── PROJECT.md          # Ta dokumentacja (v1.2)
└── air_condition.ino   # Kod bazowy (LoRa działa)
```

Koniec sekcji kontekstu. Powodzenia w kolejnej sesji!